

Java2 Day03 핵심 정리

Collection, Iterator, List/Set/Map, Stream의 핵심을 쉬운 말과 코드로 정리

Collection

여러 데이터를 담는 자료구조 묶음입니다.

Iterator

컬렉션을 같은 방식으로 하나씩 꺼내는 도구입니다.

List / Set / Map

데이터 저장 목적에 따라 선택하는 대표 자료구조입니다.

Stream

데이터를 필터, 변환, 정렬, 집계하는 흐름입니다.

1. Day03에서 배우는 Java 문법

Day03은 여러 데이터를 다루는 방법을 배웁니다. 데이터가 하나면 변수 하나로 충분하지만, 데이터가 여러 개면 **Collection**이 필요합니다. 그리고 그 데이터를 편하게 가공하려면 **Stream**을 사용합니다.

개념	쉬운 뜻	대표 코드
List	순서가 있고 중복을 허용하는 목록	ArrayList, LinkedList
Set	중복을 허용하지 않는 집합	HashSet, TreeSet
Map	key와 value를 한 쌍으로 저장	HashMap
Iterator	컬렉션 요소를 하나씩 꺼내는 표준 방법	hasNext(), next()
Stream	데이터 처리 파이프라인	filter, map, sorted, collect

자료구조 선택 기준: 순서와 중복이 중요하다면 List, 중복 제거가 중요하다면 Set, key로 빠르게 찾고 싶으면 Map을 선택합니다.

2. Collection 전체 그림

Collection은 여러 데이터를 저장하는 자료구조의 큰 묶음입니다. List와 Set은 Collection 계열이고, Map은 key-value 구조라 별도 계열로 생각하면 쉽습니다.

2-1. List

```
List<String> list = new ArrayList<>();  
list.add("one");  
list.add("two");  
list.add("two");  
  
list.forEach(item -> System.out.println(item));
```

List는 입력한 순서를 유지하고 중복도 허용합니다. 학생 목록, 게시글 목록, 주문 목록처럼 "순서가 있는 데이터"에 잘 맞습니다.

2-2. Set

```
Set<String> set = new HashSet<>();  
set.add("hi");  
set.add("hi2");  
set.add("hi2");  
  
set.forEach(item -> System.out.println(item));
```

Set은 중복을 허용하지 않습니다. 위 코드에서 "hi2"를 두 번 넣어도 실제로는 한 번만 저장됩니다.

2-3. Map

```
Map<String, String> map = new HashMap<>();  
map.put("key1", "내용1");  
map.put("key2", "내용2");  
  
String value = map.get("key1");  
System.out.println(value);
```

Map은 key로 value를 찾는 구조입니다. 회원 아이디로 회원 정보를 찾거나, 상품 코드로 상품명을 찾을 때 좋습니다.

```
for (Map.Entry<String, String> entry : map.entrySet()) {  
    String key = entry.getKey();  
    String value = entry.getValue();  
    System.out.println(key + " = " + value);  
}
```

3. Iterator

Iterator는 컬렉션의 내부 구조를 몰라도 요소를 하나씩 꺼낼 수 있게 해 주는 표준 인터페이스입니다. List든 Set이든 같은 방식으로 순회할 수 있습니다.

```

ArrayList<String> list = new ArrayList<>();
list.add("1");
list.add("2");
list.add("3");

Iterator<String> it = list.iterator();
while (it.hasNext()) {
    String item = it.next();
    System.out.println(item);
}

```

메서드	뜻
hasNext()	다음 요소가 남아 있으면 true
next()	다음 요소를 꺼냄

Java 5 이후에는 for-each문과 forEach 메서드를 많이 쓰지만, 내부적으로 "하나씩 꺼낸다"는 생각은 Iterator와 연결됩니다.

4. HashSet에서 객체 중복 판단

문자열은 같은 문자열이면 중복으로 잘 판단됩니다. 하지만 직접 만든 객체는 Java가 "무엇이 같다는 뜻인지" 모릅니다. 그래서 **equals**와 **hashCode**를 함께 오버라이드해야 합니다.

```

class User {
    String id;
    String pw;

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        User other = (User) obj;
        return Objects.equals(id, other.id)
            && Objects.equals(pw, other.pw);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, pw);
    }
}

```

equals는 "두 객체가 같은가"를 판단하고, hashCode는 해시 기반 자료구조에서 객체를 찾는 데 쓰이는 정수값입니다.

Set에 직접 만든 객체를 넣을 때: 중복 제거가 제대로 되려면 equals()와 hashCode()를 같이 재정의해야 합니다.

5. Stream이란?

Stream은 배열, List, Set 같은 여러 데이터 소스를 **같은 방식으로 처리**하게 해 주는 기능입니다. 데이터 원본을 직접 바꾸는 것이 아니라, 데이터를 흘려보내면서 필터링하고 변환하고 결과를 만듭니다.

```
String[] arr = {"one", "two", "three"};
Stream<String> stream1 = Arrays.stream(arr);

ArrayList<String> list = new ArrayList<>();
Stream<String> stream2 = list.stream();
```

Stream 특징	설명
데이터 소스를 변경하지 않음	원본 리스트나 배열 자체를 직접 수정하지 않음
일회용	한 번 최종 연산을 하면 다시 사용할 수 없음
내부 반복	반복문을 직접 쓰지 않고 Stream 내부가 반복 처리
중간 연산과 최종 연산	중간 연산은 연결, 최종 연산은 결과 생성

6. 중간 연산과 최종 연산

Stream은 보통 **중간 연산**을 여러 개 연결하고, 마지막에 **최종 연산**으로 결과를 만듭니다.

```
int[] arr = {1, 2, 3, 4, 5};

int sum = Arrays.stream(arr)
    .filter(n -> n % 2 == 1)
    .sum();
```

이 코드는 배열에서 홀수만 걸러서 합계를 구합니다. `filter`는 중간 연산이고, `sum`은 최종 연산입니다.

종류	예	뜻
중간 연산	filter	조건에 맞는 값만 통과
중간 연산	map	값을 다른 값으로 변환
중간 연산	sorted	정렬
중간 연산	distinct	중복 제거
최종 연산	forEach	하나씩 출력/처리
최종 연산	count	개수 구하기
최종 연산	collect	List, Map 등으로 모으기
최종 연산	reduce	여러 값을 하나의 값으로 줄이기

6-1. filter + forEach

```
list.stream()
    .filter(str -> str.equals("hello"))
    .forEach(str -> System.out.println(str));
```

6-2. count

```
long count = list.stream()
    .filter(str -> str.equals("hello"))
    .count();
```

6-3. collect

```
Map<String, Integer> map = customers.stream()
    .collect(Collectors.toMap(
        customer -> customer.getName(),
        customer -> customer.getAge()
    ));
```

6-4. reduce

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

int sum = numbers.stream()
    .reduce(0, (a, b) -> a + b);
```

reduce는 여러 요소를 하나의 결과로 줄입니다. 여기서는 1, 2, 3, 4, 5를 하나의 합계 15로 줄입니다.

7. Stream 문제 풀이 방향

문제 유형	사용할 연산	예
짝수만 출력	filter + forEach	<code>n -> n % 2 == 0</code>
문자열 길이 출력	map 또는 mapToInt	<code>s -> s.length()</code>
대문자 변환	map	<code>s -> s.toUpperCase()</code>
합계/평균	mapToInt, sum, average	숫자 스트림
정렬	sorted	오름차순/내림차순
중복 제거	distinct	중복 문자열 제거
객체 조건 검색	filter + map	80점 이상 학생 이름

```
List<Integer> scores = Arrays.asList(45, 80, 90, 55, 100);

scores.stream()
    .filter(score -> score >= 60)
    .forEach(System.out::println);
```

8. 마지막 핵심

주제	반드시 기억할 말
List	순서가 있고 중복을 허용한다.
Set	중복을 허용하지 않는다. 객체 중복 판단에는 equals/hashCode가 중요하다.
Map	key로 value를 찾는 구조다.
Iterator	컬렉션을 같은 방식으로 순회하기 위한 도구다.
Stream	데이터를 필터링, 변환, 정렬, 집계하는 처리 흐름 이다.